Last Week

1. [**Model**]: Change the norm clamping of the omega norm to something smoother, e.g., log-barrier loss; sigmoid/exponential + stable behavior at zero?
2. [**Distance**]: Improve the geodesic approximation in the Finsler distance computation
3. [**Loss**]: Implement Euclidean alignment among the omega vectors
4. [**Loss**]: Compute another triplet loss using the inverse triplet (p, a, n'), where the positive is now the anchor (reverse direction) -> minimize the sum of these directional losses
5. [**Experiment**]: Reduce the embedding size of the drift vector
6. [**Master Thesis**]: Updates and Proposals

Pushed to Next Week

1. [**Sanity Test**]: Devise a small experiment to test the logic behind cross-res asymmetry and camera-viewpoint bias (e.g., small dataset + simple metric).
2. [**Loss**]: Consider implementing different repulsive forces to enforce domain generalization (other than uniformity and domain loss).
   - *Idea*: Something similar to the Finsler stress loss

- Finsler might not be classification, because classification wants to be invariant
- Preserving the drift
- Does any other downstream application of reid models that is not classification

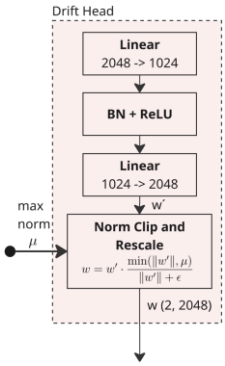
- Like training ResNet50 on ImageNet but we use it for something else

[Model] Improve Finsler Drift Head

Requirement: Model should match with Euclidean baseline when norm is zero
Suggestion: Avoid hard clamping to facilitate smooth gradients

Changes:

1. Initialized the last layer to zero
2. Removed the bias term in linear layers
3. Removed BatchNorm1d
4. Changed the clamping to something softer
   - Soft Tanh Scaling
   - Sigmoid Gated Scaling
   - Log-Barrier

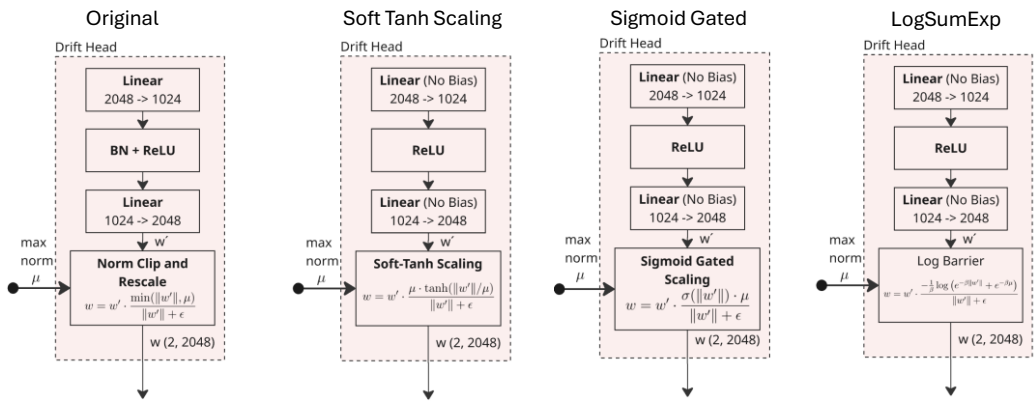


[Model] Improve Finsler Drift Head

Link to Interactive Graph

Changes:

1. Initialized the last layer to zero; Removed the bias term in linear layers + BN
2. Changed the clamping to something softer



Read up on He Initialization

- Option 1: hyperbolic tangent function to smoothly squash the norm of the drift vector so it asymptotically approaches max norm but never quite reaches it (behaves linearly for small norms).

2. Sigmoid Gated Scaling

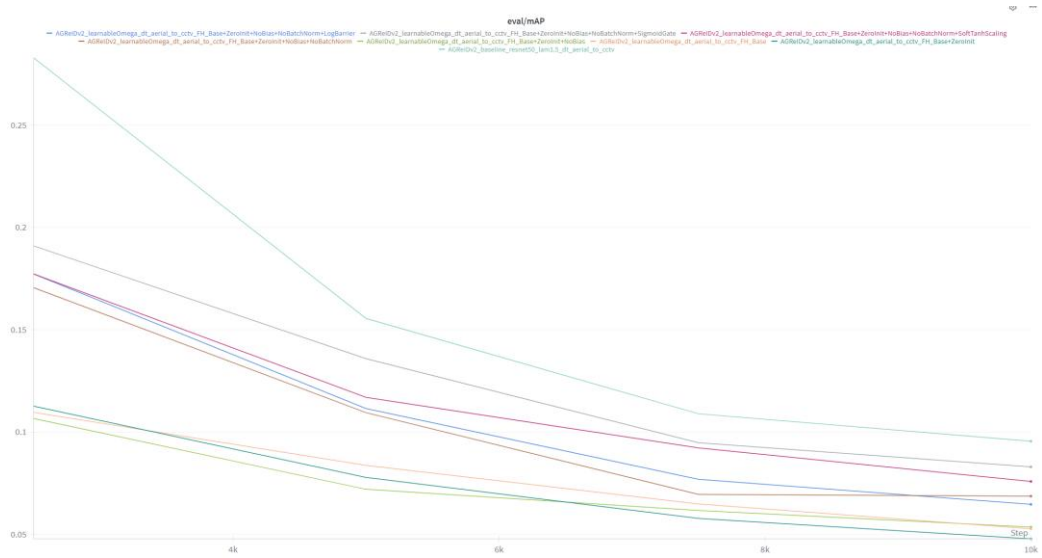
To allow the network to "decide" the magnitude more explicitly, we can gate the output. This is essentially an attention mechanism on the magnitude. Note that this doesn't strictly enforce max_norm unless you multiply by it, but it creates a smooth 0-to-max profile.

3. Log-Barrier / Soft-Maximum

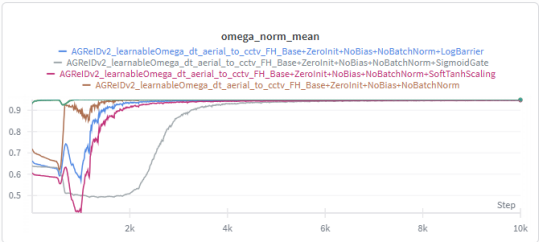
A "soft maximum" can be approximated using the LogSumExp trick or Softplus, but specifically for bounding a norm, a smooth minimum between the norm and max_norm is essentially what clamp does. Note that I found out that the mathematical expression here is wrong, hence the results shown soon are not faithful to the approach.

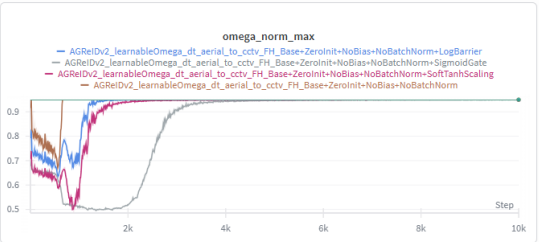
[Model] Improve Finsler Drift Head

[Model] Improve Finsler Drift Head



[Model] Improve Finsler Drift Head





[Distance] Improve the geodesic approximation in Finsler distance computation

We make two key simplifications:

- **Path Assumption**: Taking straight Euclidean line segment as the path between x and y
- **Field Assumption**: The drift vector field w changes linearly along this path

1 $$\gamma(t) = \mathbf{x}_{id} + t(\mathbf{y}_{id} - \mathbf{x}_{id}) \qquad \dot{\gamma}(t) = \mathbf{y}_{id} - \mathbf{x}_{id}$$

2 $$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \int_0^1 \langle \omega(\gamma(t)), \dot{\gamma}(t) \rangle dt$$

3 $$\omega(\gamma(t)) \approx (1-t)\mathbf{x}_\omega + t\mathbf{y}_\omega \qquad \mathcal{A}(\mathbf{x}, \mathbf{y}) \approx \int_0^1 \langle (1-t)\mathbf{x}_\omega + t\mathbf{y}_\omega, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle dt$$

$$\int_0^1 (1-t)dt = \frac{1}{2}, \quad \int_0^1 t dt = \frac{1}{2}$$

4 $$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle$$

$$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \frac{1}{2}\left[(\mathbf{x}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{x}_\omega \cdot \mathbf{x}_{id}) + (\mathbf{y}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{y}_\omega \cdot \mathbf{x}_{id})\right]$$

Symmetrical Trapezoidal Approximation

1. **Define the Path**
   - We define the path $\gamma(t)$ in the identity feature space from x_{id} to y_{id} , parameterized by $t \in [0, 1]$.
   - The velocity vector (tangent to the path) is constant:

2. **Define the Integral**
   The asymmetric component $A(x, y)$ of the Randers metric is defined as the path integral of the drift one-form ω acting on the velocity vector.

3. **Linear Interpolation (Trapezoidal Approximation)**
   By assuming the drift vectors change linearly, we approximate $\omega(\gamma(t))$ as a weighted sum of the drift at start (x_ω) and end (y_ω).
   - Substituting this back into the integral
   - Since the velocity term is constant with respect to t, we integrate the scalar coefficients

4. **Step 4: The Closed Form Solution**
   - This simplifies the integral to the dot product of the **average drift vector** and the displacement vector.
   - Finally, we expand this dot product into the four specific terms

5. X and y have the same dimensions

Randers distance of the arc circle?

[Distance] Improve the geodesic approximation in Finsler distance computation

We make two key simplifications:

- **Path Assumption**: The geodesic is approximated as a straight Euclidean line segment
- **Field Assumption**: The drift vector field w changes linearly along this path

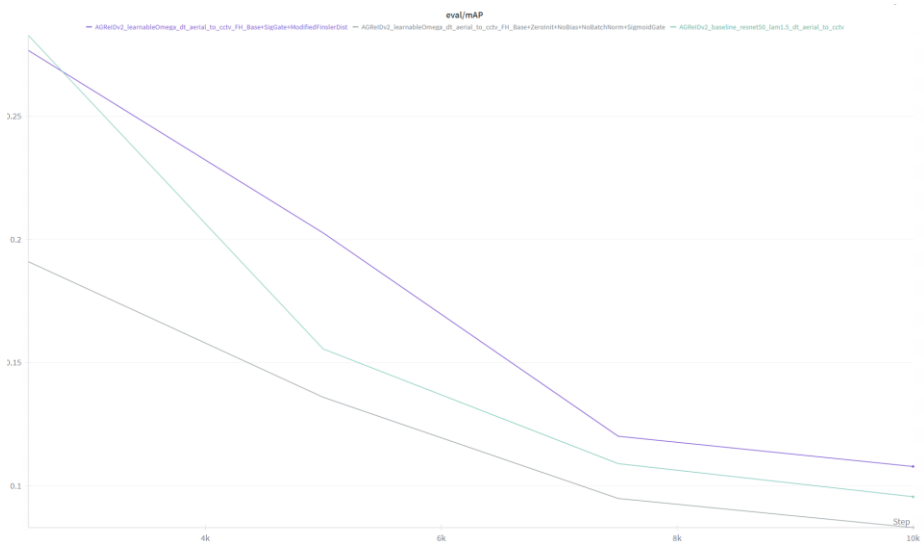
$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\mathrm{id}} - \mathbf{y}_{\mathrm{id}}\|_2 + \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle$$

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\mathrm{id}} - \mathbf{y}_{\mathrm{id}}\|_2 + \frac{1}{2}\left[(\mathbf{x}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{x}_\omega \cdot \mathbf{x}_{id}) + (\mathbf{y}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{y}_\omega \cdot \mathbf{x}_{id})\right]$$

[Distance] Improve the geodesic approximation in Finsler distance computation

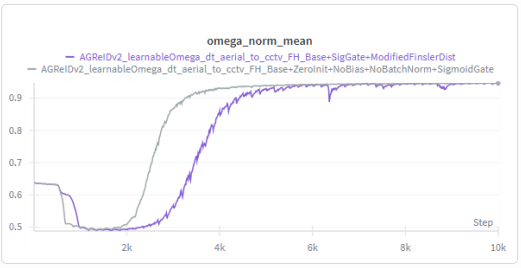
Do cross validation on the evaluation set (LOOCV)

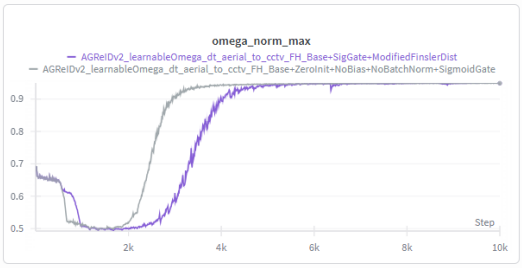
[Distance] Improve the geodesic approximation in Finsler distance computation



Do cross validation on the evaluation set (LOOCV)

[Distance] Improve the geodesic approximation in Finsler distance computation





Do cross validation on the evaluation set (LOOCV)

[Loss]: Implement Euclidean alignment among the omega vectors

Previously:

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \mathbf{x}_\omega \cdot (\mathbf{y}_{\text{id}} - \mathbf{x}_{\text{id}})$$

$$\mathcal{L}_{\text{align}} = \sum_{(i,j)\in\mathcal{I}_{\text{pos}}} \bar{w}_{ij} \|\tilde{\mathbf{f}}_{i,\text{id}} - \mathbf{f}_{j,\text{id}}\|_2^2$$

Now:

$$\mathcal{L}_{\text{align}} = \sum_{(i,j)\in\mathcal{I}_{\text{pos}}} \bar{w}_{ij} \|\tilde{\mathbf{f}}_{i,\text{id}} - \mathbf{f}_{j,\text{id}}\|_2^2 + \sum_{(i,j)\in\mathcal{I}_{\text{pos}}} \bar{w}_{ij} \|\tilde{\mathbf{f}}_{i,\omega} - \mathbf{f}_{j,\omega}\|_2^2$$

1. When Drift Represents "Static" Domain Characteristics

If the drift vector (w) is intended to capture the inherent style of the camera or scene (e.g., lighting, background, camera angle) rather than the specific noise added by data augmentation, aligning it forces the model to produce the **same style embedding** regardless of whether the input image is weakly or strongly augmented.

- **Without Drift Alignment:** The model might encode the strong augmentation (e.g., heavy color jitter or cutout) *into* the drift vector. This means $w_{strong} \neq w_{weak}$.
- **With Drift Alignment:** You force $w_{strong} \approx w_{weak}$ .This ensures the drift vector purely captures the underlying domain style (e.g., "This is Camera 1") and ignores the transient augmentation noise.

2. To Prevent Information Leakage

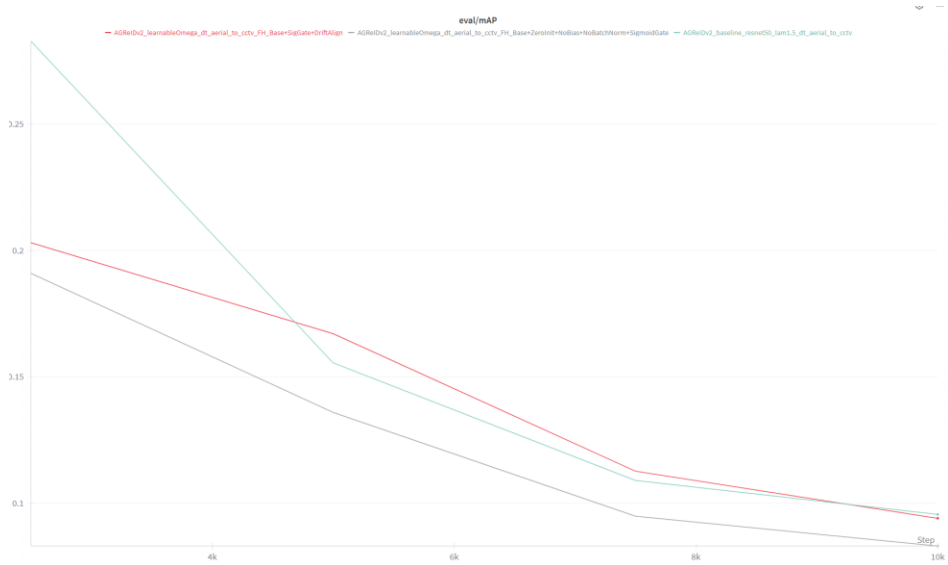
If the drift vector is allowed to change freely between augmentations, the network might "hide" identity information inside the drift vector to satisfy the reconstruction or classification losses easier, rather than disentangling them properly. Forcing consistency (alignment) acts as a regularizer, making the decomposition into $Identity + Drift$ more robust.

3. In Contrast to "Drift as Noise"

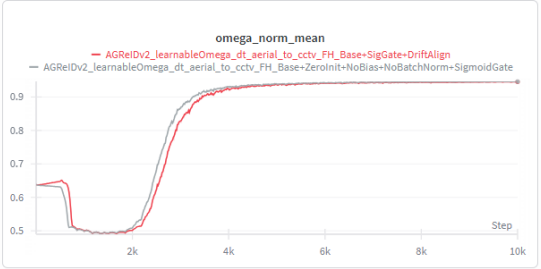
You would **NOT** align the drift vector if you explicitly want the drift component to capture the augmentation itself (i.e., if you treat the augmentation as a domain shift). In that case, w_{strong} *should* be different from w_{weak} .However, in most ReID contexts involving "f_w" (weak) and "f_s" (strong), you usually want the features (both identity and global style) to remain consistent for the same image instance

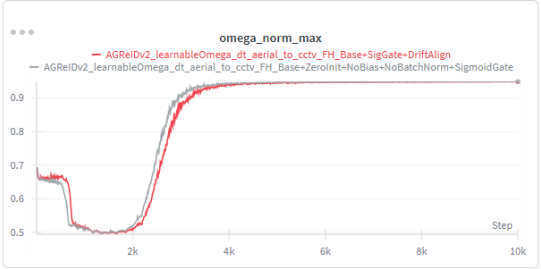
[Loss]: Implement Euclidean alignment among the omega vectors

[Loss]: Implement Euclidean alignment among the omega vectors



[Loss]: Implement Euclidean alignment among the omega vectors





[Loss]: Implement reverse triplet loss

Hypothesis: Minimizing the sum of these directional losses shapes the embedding space such that the drift "w" minimizes the retrieval error in both directions independently.

Before

$$\mathcal{L}_{\text{triplet,BH}} = \frac{1}{B}\sum_{i=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(i)}\{d_f(x_i, x_p)\} - \min_{n\in\mathcal{N}(i)}\{d_F(x_i, x_n)\} + \alpha\right)$$

Now

$$\mathcal{L}_{\text{bi-triplet}} = \frac{1}{2}\left(\mathcal{L}_{\text{tri,f}}(i) + \mathcal{L}_{\text{tri.b}}(i)\right)$$

$$\mathcal{L}_{\text{tri,f}}(i) = \frac{1}{B}\sum_{j=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(j)}\{d_F(x_j, x_p)\} - \min_{n\in\mathcal{N}(j)}\{d_F(x_j, x_n)\} + m\right)$$

$$\mathcal{L}_{\text{tri,b}}(i) = \frac{1}{B}\sum_{j=1}^{B} \max\left(0, \max_{p\in\mathcal{P}(j)}\{d_F(x_p, x_j)\} - \min_{n\in\mathcal{N}(j)}\{d_F(x_n, x_j)\} + m\right)$$

Intuition: Since $d(a, b) \neq d(b, a)$,optimizing $L(a, p, n)$ only ensures that a can retrieve p. It guarantees nothing about p retrieving a. Minimizing the sum ensures the drift component facilitates retrieval in *both* directions. This aligns with the hypothesis about shaping the drift w.

Standard Forward Loss (mat_dist):

We iterate through each row i (treating i as the **anchor**).
We search the columns to find a positive p and negative n.
We minimize: $d(x_i, x_p) - d(x_i, x_n)$.
Meaning: "Make anchor i closer to its hardest positive neighbor than to its nearest impostor."

Inverse Loss (mat_dist.t()):

We transpose the matrix, so row j now contains distances *from* all other points *to* j: $[d, (x_0, x_j), d(x_1, x_j) \ldots]$.
We iterate through each row j of this transposed matrix. Here, j **is the target**, and the "columns" (which were originally rows) are the potential

queries.

_batch_hard selects the index q (query) that maximizes $d(x_q, x_j)$ among positives, and index k that minimizes $d(x_k, x_j)$ among negatives.

Meaning: "Make target j be retrieved by its 'worst' aligned positive query q better than it is retrieved by its 'best' aligned negative query k."

Why this distinction matters

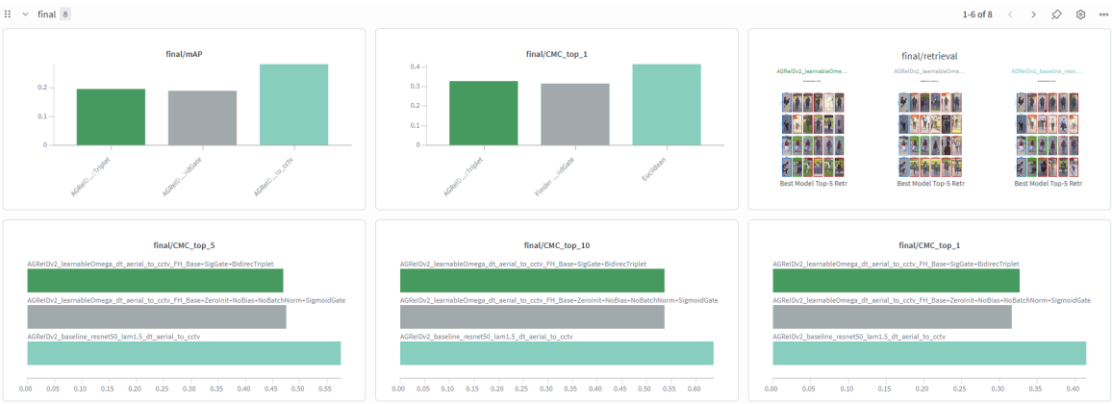
In a symmetric Euclidean space, $d(x_q, x_j) = d(x_j, x_q)$ .The set of distances is identical, but the *mining* might select different indices.

In the forward pass, for anchor A, maybe B is the hardest positive ($d(A, B)$ is max).

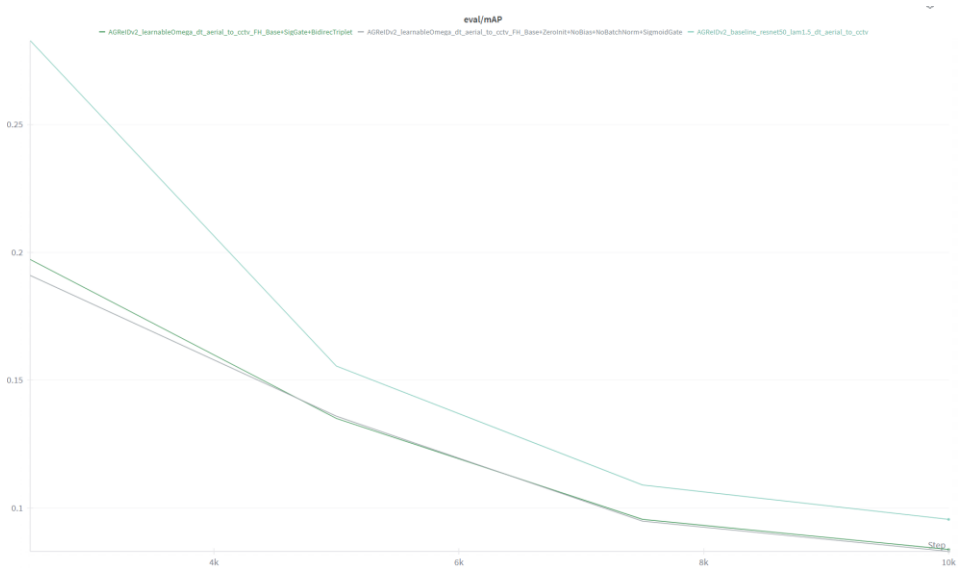
In the inverse pass, for target A, maybe C is the hardest query ($d(C, A)$ is max).

- Look into the learning rate scheduling, should decrease instead of increase
- Inside the parentheses cannot be the same running variable as in the sum

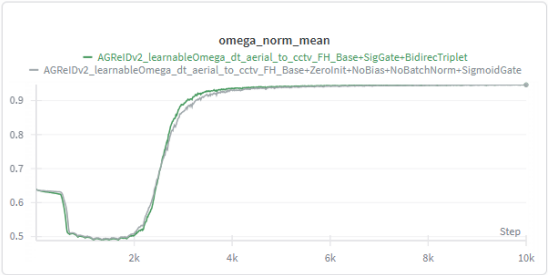
[Loss]: Implement reverse triplet loss

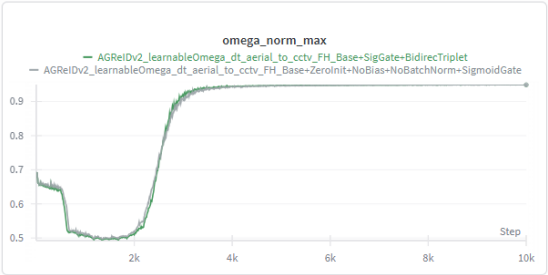


[Loss]: Implement reverse triplet loss



[Loss]: Implement reverse triplet loss





[Experiment] Reduce the embedding size of the drift vector

Previously:

$$d_F(\mathbf{x}, \mathbf{y}) = \underbrace{\|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2}_{\text{identity distance}} + \underbrace{\mathbf{x}_\omega \cdot (\mathbf{y}_{\text{id}} - \mathbf{x}_{\text{id}})}_{\text{viewpoint-dependent drift}}$$

Now:

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \sum_{k=1}^{S} \mathbf{x}_\omega^{(k)} (\mathbf{y}_{\text{id}}^{(k)} - \mathbf{x}_{\text{id}}^{(k)})$$

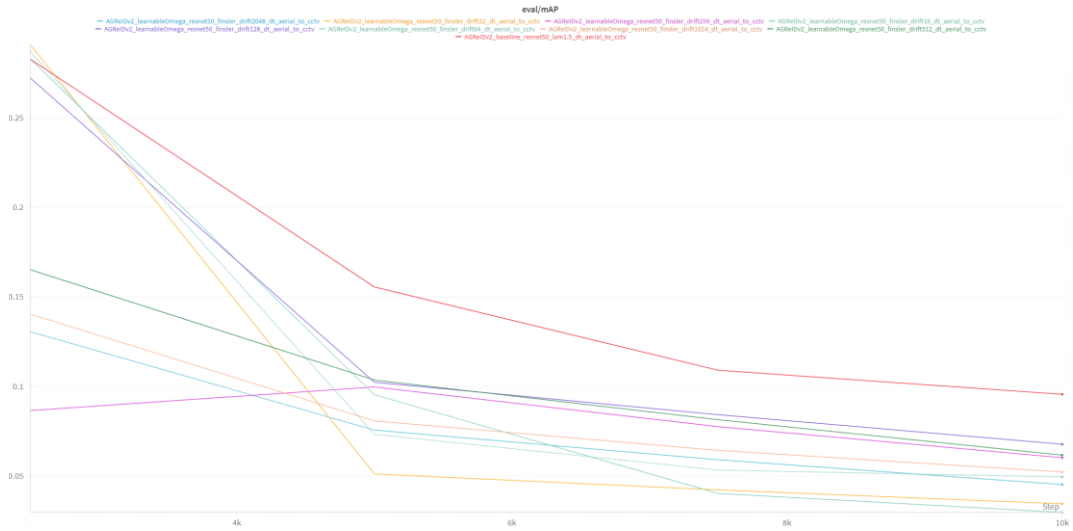
$$S = \min(\dim(\mathbf{x}_{\text{id}}), \dim(\mathbf{x}_\omega))$$

Note that this experiment was not made in tandem with the previous. It was done as the first one, as I didn't know how long the sweep was going to take

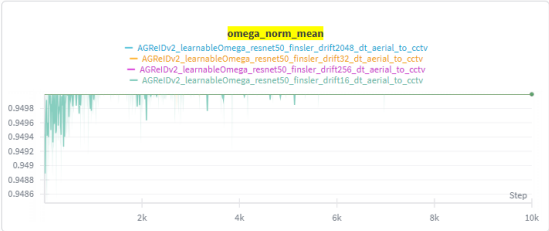
Multiply the sum by N/S, where N is the embedding size of x and y

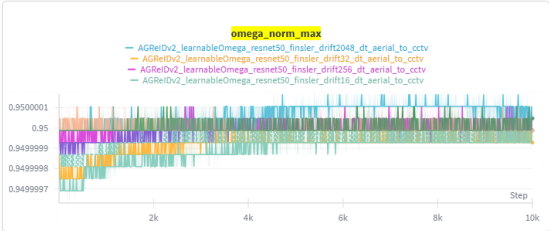
[Experiment] Reduce the embedding size of the drift vector

[Experiment] Reduce the embedding size of the drift vector



[Experiment] Reduce the embedding size of the drift vector





Today

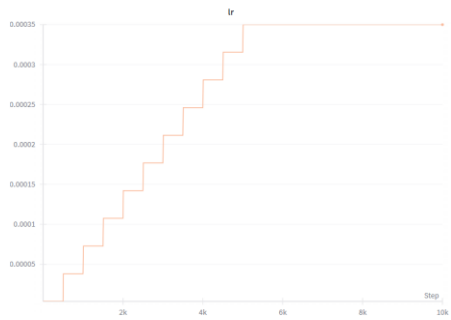
1. [**Training**]: Fix the learning rate scheduler such that it anneals over time instead of increasing; redo all the experiments
2. [**Experiment**]: Combine all of the improvements in this week into one model, going step-by-step to understand the contribution of each modification
   - Do cross-validation on the evaluation/test set to rule out random seeding behaviors
3. [**Model**]: ~~Fix the LogSumExp activation function / approximation and re-run the test~~
4. [**Theory/Experiment**]: Brainstorm a better approximation of the Rander's drift vector compared to the symmetric trapezoidal approximation
5. [**Theory/Experiment**]: Consider the improvements in reducing the drift vector dimension as a potential scaling problem (multiply by N/S) – substantiate this argument and test it out

Put on Hold

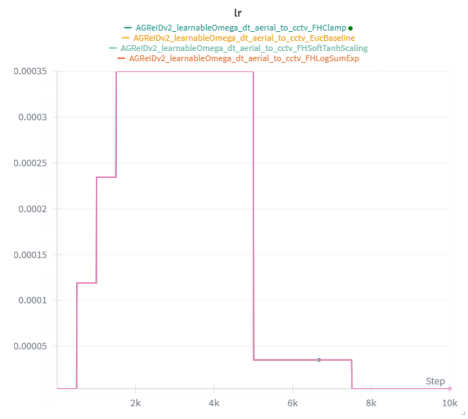
1. [**Loss**]: Consider implementing different repulsive forces to enforce domain generalization (other than uniformity and domain loss). *Idea*: Something similar to the Finsler stress loss
2. [**Sanity Test**]: Devise a small experiment to test the logic behind cross-res asymmetry and camera-viewpoint bias (e.g., small dataset + simple metric).

[Training] Modified Learning Rate Scheduler

Before



Now



- Before we used an unscaled learning rate schedule where the default step-decay milestones ($M = [30, 50]$) fall outside the truncated 20-epoch limit, resulting in an absence of learning rate decay. I've proportionally scaled both the linear warmup phase and the milestone decay steps to strictly fit within the 10k step horizon ($E = 20$), restoring the required gradient attenuation.

- To mathematically isolate the contributions of the proposed Finsler manifold formulation from hyperparameter-induced variance, the canonical BAU Euclidean ResNet baseline is optimized and evaluated under this exact identical compressed 20-epoch schedule.

- Compressing the optimization budget is theoretically suboptimal for deep metric learning manifolds. Truncating the training forces the network to terminate during a high-variance transient phase, limiting the capacity for fine-grained feature disentanglement and preventing the embeddings from reaching a stable, uniform distribution.

- This truncated schedule is a strictly temporary heuristic. It deliberately sacrifices asymptotic convergence bounds to maximize experimental

throughput, enabling rapid iteration across architectural variants and Randers metric parameterizations during the preliminary search phase.

- **Canonical Scaling Protocol:** Upon empirical validation of the optimal architectural formulation, the training protocol will definitively revert to the canonical $E = 60$ regime. The final evaluation will utilize the complete multi-domain composite dataset (Market1501, MSMT17, CUHK-SYSU, AGReID) to ensure robust gradient convergence, minimize source-domain overfitting, and establish rigorous state-of-the-art validation.

EXTRAS

In the context of deep neural network optimization and metric learning, the **transient phase** (often referred to as the early-training transient regime) denotes the initial and intermediate stages of gradient descent where the learning rate remains at its peak or warmup maxima. Mathematically and geometrically, this phase is characterized by:

- **High-Variance Gradient Updates:** The large step sizes prevent the optimizer from settling into sharp local minima. Instead, the parameters traverse the macro-structure of the loss landscape.
- **Coarse-Grained Embedding Displacements:** In the latent metric space (e.g., the hypersphere or Finsler manifold), the representations undergo massive structural shifts. The repulsive forces of the triplet or contrastive loss push dissimilar instances apart, but the high kinetic energy of the optimizer prevents fine-grained, exact disentanglement.
- **Absence of Asymptotic Uniformity:** To achieve a theoretically optimal representation, the feature vectors must be distributed uniformly across the latent manifold (maximizing entropy and minimizing hyperspherical energy). During the transient phase, the continuous injection of optimization noise—driven by the high learning rate and mini-batch stochasticity—causes the embeddings to constantly oscillate.

Only after the learning rate decays (entering the *stationary* or *asymptotic phase*) can the network execute micro-adjustments, allowing the contrastive gradients to strictly enforce fine-grained feature disentanglement and uniform packing without overshooting. Truncating the training before this decay forces the model to output representations that are mathematically unstable and strictly sub-optimal for open-set retrieval.

Primary Citation for DML Uniformity and Convergence:

Wang, T., & Isola, P. (2020). Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere. *In International Conference on Machine Learning (ICML)*.

- *Relevance:* Wang and Isola mathematically formalize that optimizing the contrastive objective requires asymptotic convergence to minimize the hyperspherical energy (the "uniformity" metric). Truncating optimization prematurely prevents the embeddings from satisfying this uniformity bound, leaving the latent space highly localized and poorly generalized.

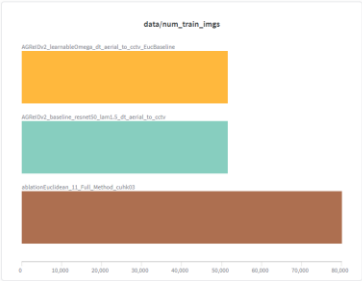
Primary Citation for Transient Phase Dynamics and DML Generalization:
Roth, K., Milbich, T., Sinha, S., Gupta, P., Ommer, B., & Cohen, J. P. (2020). Revisiting Training Strategies and Generalization Performance in Deep Metric Learning. *In International Conference on Machine Learning (ICML)*.

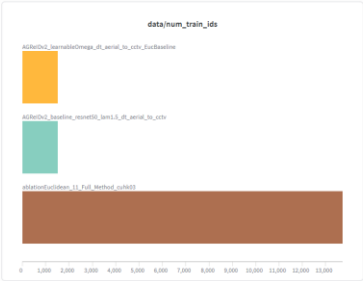
- *Relevance:* Roth et al. explicitly investigate the impact of training durations, batch scheduling, and learning rate decays in Deep Metric Learning. They demonstrate that DML models evaluated during early or truncated epochs exhibit high-variance representations that fail to capture diverse, fine-grained visual characteristics. The paper strictly proves that stable, disentangled feature representations strictly require completing the full optimization schedule, validating your critique of the 20-epoch truncation.

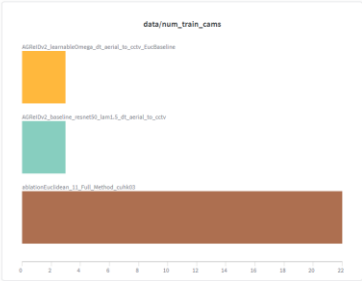
Reference for the Terminology "Transient Phase":
Lewkowycz, A., Bahri, Y., Roelofs, R., Alonso, E., & Sharma, V. (2020). The large learning rate phase of deep learning: the catapult mechanism. *arXiv preprint arXiv:2003.02218*.

- *Relevance:* This work formalizes the "transient phase" of optimization, proving that while high initial learning rates are necessary to escape sharp, poor-generalizing basins, the network must subsequently transition out of this high-variance transient phase via step-decay to achieve optimal feature separation.

[Training] Why only 20 epochs?







[Test] Why Re-ID does not support conventional cross-validation

- **The Open-Set Paradigm**: Train and test identities must remain strictly disjoint to evaluate true generalization, not memorization.
- **Data Leakage**: Standard k-fold cross-validation randomly splits images, leaking training identities into validation folds and violating open-set rules [1].
- **Complex Camera Topology**: ReID requires matching a *Query* (1 camera) against a *Gallery* (different cameras). Dynamically generating valid cross-camera splits per fold is error-prone.
- **Standardization for Benchmarking**: Metrics (mAP, Rank-1) are extremely sensitive to the number of gallery distractors. A single, fixed split is mandatory for fair model comparison [2].

***Alternatively*:** We can train the final models with different random seeds and average out the metrics (like we did before)

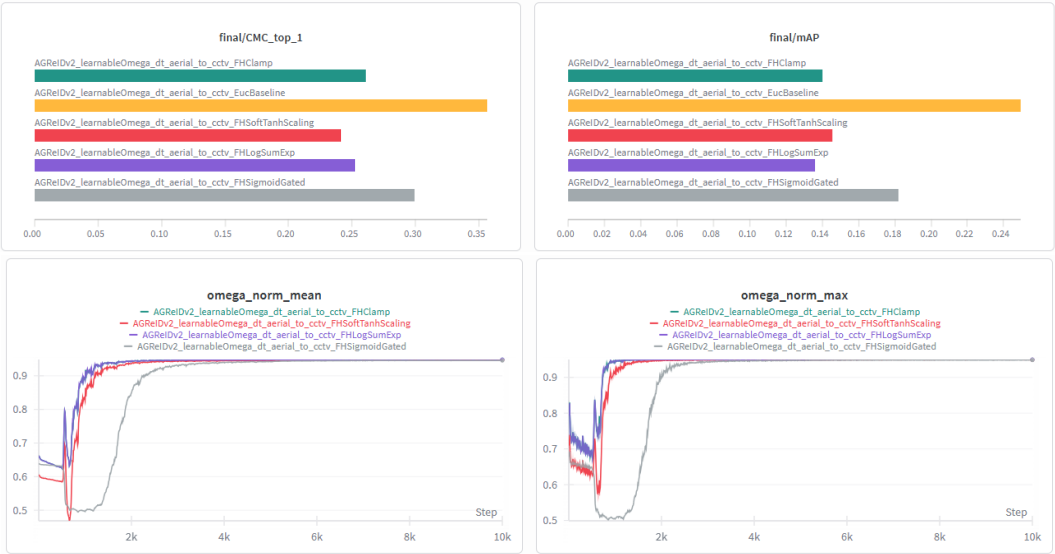
[1] Ye et al., "Deep Learning for Person Re-identification: A Survey and Outlook" (IEEE TPAMI 2021).
[2] Zheng et al., "Scalable Person Re-identification: A Benchmark" (ICCV 2015).

- Applying cross validation to Person Re-Identification breaks the fundamental rules of the task.
- ReID is about metric learning—teaching a model *how* to compare people, not *who* they are. If we use standard cross-validation, the same person appears in both training and testing. The model just memorizes their specific shirt or appearance, which artificially inflates the score and ruins our ability to test on unseen people.
- 'Why not just split the folds by identity?' To prove a ReID model works, we must guarantee we are matching a person from Camera A to Camera B. Creating perfectly balanced, unbiased Query and Gallery splits across multiple folds that respect this cross-camera rule is mathematically messy and introduces severe variance.
- Furthermore, the primary metrics—Rank-1 accuracy and Mean Average Precision (mAP)—are highly dependent on the gallery size. Finding one person among 1,000 distractors is much easier than finding them among 20,000. If folds have different gallery distributions, the scores become meaningless.

Conclusion: Because of this, since the landmark Market-1501 paper in 2015, the community universally agreed to use exactly one fixed, heavily standardized data

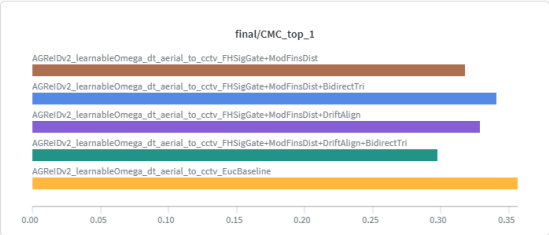
split per dataset. This ensures absolute fairness when benchmarking models like we see in codebases like BAU and FastReID.

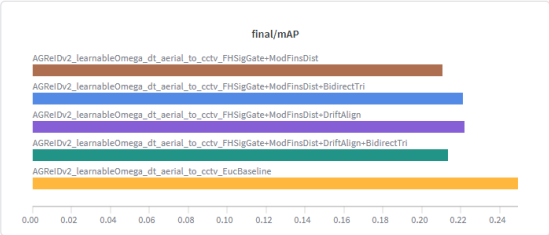
[Model] Improving the Finsler Drift Head (Pt. 2)

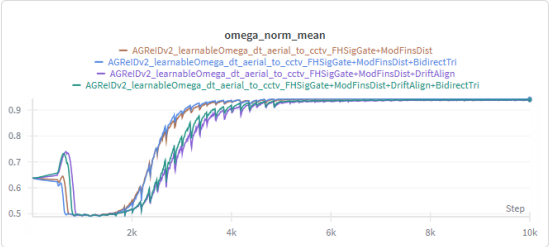


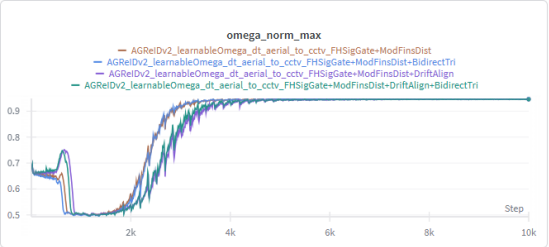
Moving forward, we are benchmarking models that have the sigmoid gate implemented in the Finsler head

[Model+Loss] Combining all of the Improvements









[Model+Loss] Combining all of the Improvements

ModFistDist+DriftAlign+BidirectTri

EucBaseline

[Model+Loss] Combining all of the Improvements

ModFistDist

ModFistDist + BidirectTri

ModFistDist + DriftAlign

[Distance] Improve the geodesic approximation in Finsler distance computation

We make two key simplifications:

- **Path Assumption**: Taking straight Euclidean line segment as the path between x and y
- **Field Assumption**: The drift vector field w changes linearly along this path

1 $$\gamma(t) = \mathbf{x}_{id} + t(\mathbf{y}_{id} - \mathbf{x}_{id}) \qquad \dot{\gamma}(t) = \mathbf{y}_{id} - \mathbf{x}_{id}$$

2 $$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \int_0^1 \langle \omega(\gamma(t)), \dot{\gamma}(t) \rangle dt$$

3 $$\omega(\gamma(t)) \approx (1-t)\mathbf{x}_\omega + t\mathbf{y}_\omega \qquad \mathcal{A}(\mathbf{x}, \mathbf{y}) \approx \int_0^1 \langle (1-t)\mathbf{x}_\omega + t\mathbf{y}_\omega, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle dt$$

$$\int_0^1 (1-t)dt = \frac{1}{2}, \quad \int_0^1 t dt = \frac{1}{2}$$

4 $$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle$$

$$\mathcal{A}(\mathbf{x}, \mathbf{y}) = \frac{1}{2}\left[(\mathbf{x}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{x}_\omega \cdot \mathbf{x}_{id}) + (\mathbf{y}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{y}_\omega \cdot \mathbf{x}_{id})\right]$$

Symmetrical Trapezoidal Approximation

1. **Define the Path**
   - We define the path $\gamma(t)$ in the identity feature space from x_{id} to y_{id} , parameterized by $t \in [0, 1]$.
   - The velocity vector (tangent to the path) is constant:

2. **Define the Integral**
   The asymmetric component $A(x, y)$ of the Randers metric is defined as the path integral of the drift one-form ω acting on the velocity vector.

3. **Linear Interpolation (Trapezoidal Approximation)**
   By assuming the drift vectors change linearly, we approximate $\omega(\gamma(t))$ as a weighted sum of the drift at start (x_ω) and end (y_ω).
   - Substituting this back into the integral
   - Since the velocity term is constant with respect to t, we integrate the scalar coefficients

4. **Step 4: The Closed Form Solution**
   - This simplifies the integral to the dot product of the **average drift vector** and the displacement vector.
   - Finally, we expand this dot product into the four specific terms

5. X and y have the same dimensions

Randers distance of the arc circle?

[Distance] Improve the geodesic approximation in Finsler distance computation

We make two key simplifications:

- **Path Assumption**: The geodesic is approximated as a straight Euclidean line segment
- **Field Assumption**: The drift vector field w changes linearly along this path

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\mathrm{id}} - \mathbf{y}_{\mathrm{id}}\|_2 + \langle \frac{\mathbf{x}_\omega + \mathbf{y}_\omega}{2}, \mathbf{y}_{id} - \mathbf{x}_{id} \rangle$$

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\mathrm{id}} - \mathbf{y}_{\mathrm{id}}\|_2 + \frac{1}{2}\left[(\mathbf{x}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{x}_\omega \cdot \mathbf{x}_{id}) + (\mathbf{y}_\omega \cdot \mathbf{y}_{id}) - (\mathbf{y}_\omega \cdot \mathbf{x}_{id})\right]$$

[Model]: Refining the Rander's Drift: Spherical Trapezoidal Approximation

Recall: The *Symmetric Trapezoidal Approximation* assumed a linear Euclidean path between embeddings x and y.

$$\gamma(t) = x + t(y - x)$$

The Flaw: L_2-normalized identity features strictly reside on a hypersphere (S^{d-1}).

- Thus, $\| x \|_2 = 1$ and $\| y \|_2 = 1$.
- A straight Euclidean line cuts through the *interior* of the sphere, leaving the valid data manifold.

Defining the Geodesic Path: Integrate the drift vector ω along the true geodesic (a great circle arc).

- The angle between them: $C = \cos(\theta) = \langle x, y \rangle$

Deriving the Spherical Tangents:
To integrate the drift vector ω along this arc, we need the path's tangent vectors. We find these by projecting the destination point onto the tangent space of the origin point ($T_x S^{d-1}$).

- *Tangent at x pointing towards y:* $v_x = y - proj_x(y) = y - \langle x, y \rangle x = y - Cx$
- *Tangent at y arriving from x:* $v_y = Cy - x$

The Final Asymmetry Term:

$$\mathcal{A}(\mathbf{x}, \mathbf{y}) \approx \frac{1}{2}\left[\langle \mathbf{x}_\omega, \mathbf{y}_{\text{id}} - C\mathbf{x}_{\text{id}} \rangle + \langle \mathbf{y}_\omega, C\mathbf{y}_{\text{id}} - \mathbf{x}_{\text{id}} \rangle\right]$$

Connecting to previous slides:

- Previousy, we introduced the Symmetric Trapezoidal Approximation to compute the Randers distance.
- To make the integral tractable, we assumed the shortest path between query x and gallery y was a straight Euclidean line.

Identifying the theoretical flaw:

- However, because our network uses L_2 normalization on the identity features, our valid embedding space is not flat—it is a hypersphere S^{d-1}
- By integrating along a straight line $\gamma(t) = x + t(y - x)$,the midpoint of our path has a norm less than 1 ($\| \frac{x+y}{2} \| < 1$).
- We are effectively evaluating the drift vector ω in the empty interior of the sphere, where the network has never been trained to map features.

The Geodesic Fix:

- To fix this, we must integrate along the true shortest path on a hypersphere: a great circle arc.
- The angle between them is given by their cosine similarity: $C = \cos(\theta) = \langle x, y \rangle$

Deriving Spherical Tangents

- Instead of the constant Euclidean velocity $y - x$, we calculate the unnormalized tangent vectors along the spherical arc.
- At point x, the tangent pointing towards y is $v_x = y - \cos(\theta)x$. We subtract the component of y that is parallel to x.
- At point y, the tangent arriving from x is $v_y = y\cos(\theta) - x$

The Final Asymmetry Term:

- Substituting these spherical tangents into the trapezoidal rule yields our new, manifold-aware asymmetry penalty

Why this improves performance:

- By projecting the path tangents onto the hypersphere, we ensure that the asymmetric penalty is calculated strictly using valid, on-manifold feature representations.
- This prevents the distance metric from being distorted by off-manifold 'shortcuts', leading to a much more accurate estimation of the information degradation between the aerial and ground views.

Extras

3. How it changes the Integral

Because the new velocity $\dot{\gamma}(t)$ is not constant, we cannot easily do what Step 3 (pulling $y_{id} - x_{id}$ out of the integral).

Instead, we approximate the integral $\int_0^1 \langle \omega'(t)\dot{\gamma}(t)\rangle dt$ by averaging the dot product at the start and the end (this is where the Trapezoidal Rule explicitly comes in):

At $t = 0$**:** $\langle \omega'(0)\dot{\gamma}(0)\rangle = \langle x_{\omega'}\, y_{id} - Cx_{id}\rangle$

At $t = 1$**:** $\langle \omega'(1)\dot{\gamma}(1)\rangle = \langle y_{\omega'}\, Cy_{id} - x_{id}\rangle$

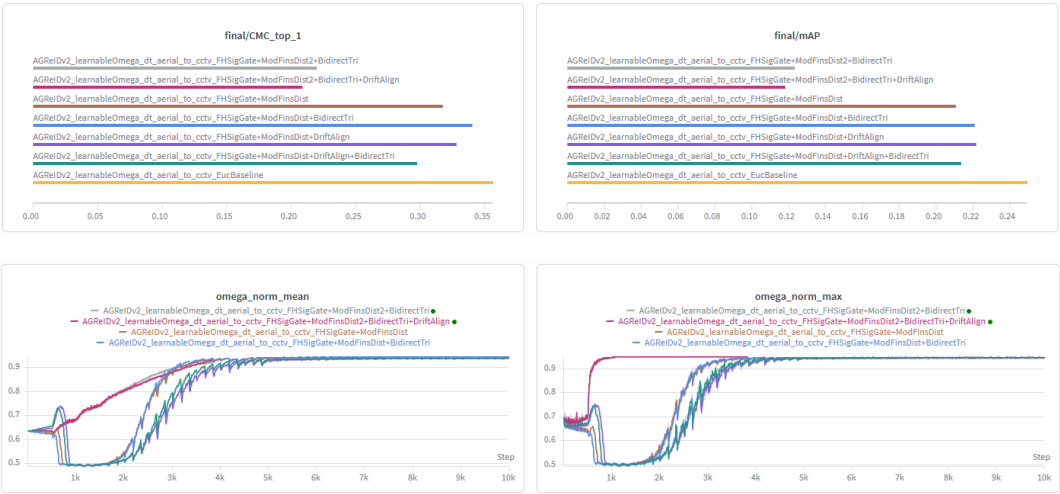
Theoretical Substantiation (if asked):

On Hyperspherical embeddings: "As shown by Wang & Isola (ICML 2020) in their work on alignment and uniformity, L_2 normalized representation learning strictly confines meaningful semantic interpolation to the surface of the hypersphere."

On Geodesics: "Standard Riemannian geometry (e.g., Lee's *Introduction to Riemannian Manifolds*) dictates that the tangent space T_xS^{d-1} is the orthogonal complement to x. The projection of y onto T_xS^{d-1} is exactly $y - \langle x, y\rangle x$,which gives us our corrected velocity vectors."

Compute the geodesic path
Slerp explicit formula that gives you the location of the trajectory – explicit formula for the trajectory
If the trajectory is on a sphere, the omegas are supposed to be tangent to the hypersphere -> learn omega on the tangent plane? How do we enforce that
Identity – XX^t is on the tangent of the sphere, where x is normalized

[Model]: Refining the Rander's Drift: Spherical Trapezoidal Approximation



Observation: We see a distinct drop in mAP and Rank-1 accuracy, and the latent space clusters are noticeably less aligned than our initial Euclidean approximation.

Analysis (The "Vanishing Asymmetry" Problem): While mathematically correct, the spherical projection introduces a dynamic scaling factor via the cosine similarity (C).

- When features belong to the same identity and get close ($C \to 1$), the tangents approach $y - x$ and $y - x$, causing the asymmetry to vanish.
- This prevents the drift vector from applying fine-grained corrections within tight clusters, leading to the observed cluster misalignment.

Analysis (Gradient Entanglement):

- In the Euclidean approximation, the gradient of the asymmetry term with respect to x was relatively straightforward.
- Now, the asymmetry term is heavily multiplied by $C = \langle x, y \rangle$ .This means the network can minimize the asymmetric triplet loss by simply manipulating the cosine similarity (pushing identity features apart) rather than learning a meaningful drift vector ω.

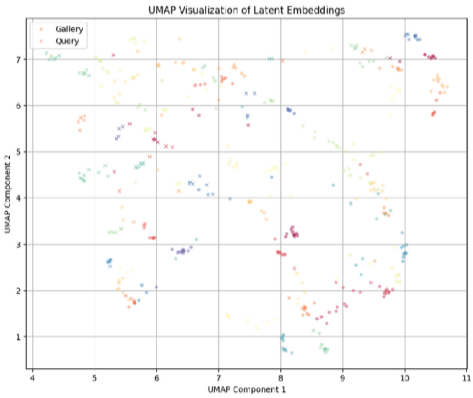
Observation: The ω (drift) norm and mean curves take a completely different shape compared to previous runs.
Analysis (Restricted Drift Growth): Because the self-projection terms ($\langle \omega_{x'} x \rangle$) are now scaled by C, the effective magnitude of the drift vector is suppressed when comparing distant domains ($C \approx 0$).

- The network struggles to establish a consistent magnitude for ω because its impact on the total distance is constantly fluctuating based on the angle between the identity features. This leads to erratic or stunted norm curves as the drift head fails to find a stable optimization trajectory.

[Model]: Refining the Rander's Drift: Spherical Trapezoidal Approximation

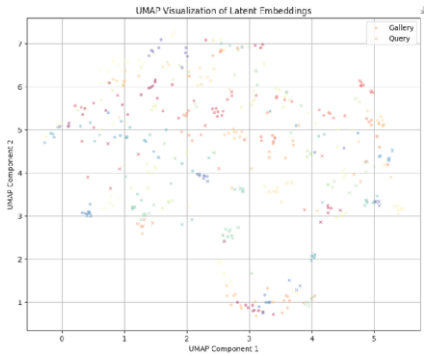
ModFistDist2+BidirectTri

ModFistDist+BidirectTri

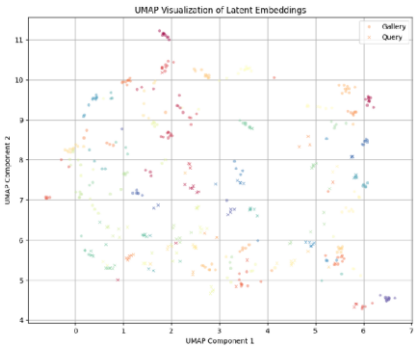


[Model]: Refining the Rander's Drift: Spherical Trapezoidal Approximation

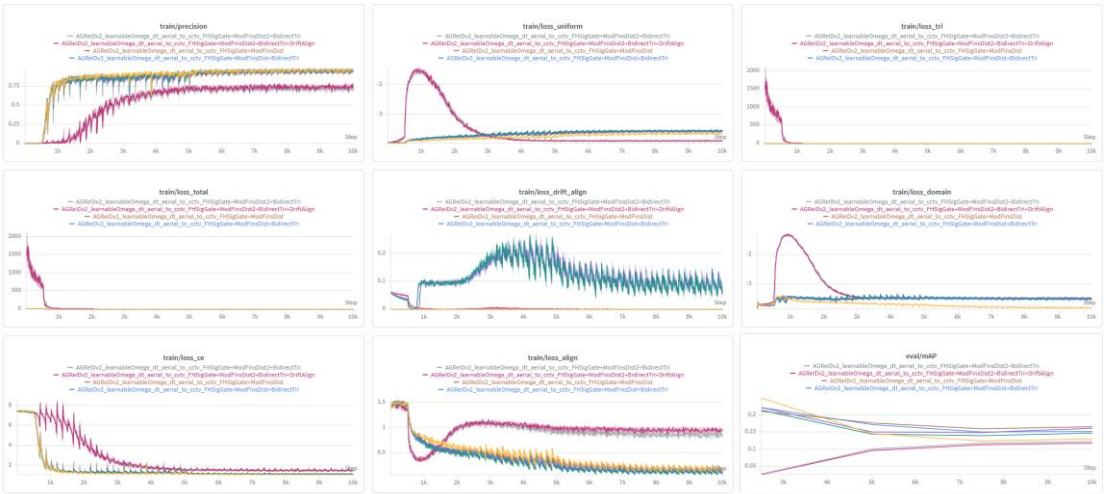
ModFistDist2+BidirectTri+DriftAlign



ModFistDist+BidirectTri+DriftAlign



[Model]: Refining the Rander's Drift: Spherical Trapezoidal Approximation



Triplet & Alignment Loss Dynamics

- **Observation:** The triplet loss starts exceptionally high, and the alignment loss (which was previously stable) begins to increase after an initial period.
- **Analysis (The Identity-Drift Conflict):** The high initial triplet loss shows that the C-scaled asymmetry severely disrupts the initial relative distances of the batch.
- **Analysis (Why Alignment Increases):** The rising alignment loss is the smoking gun.
    - Recall that our alignment loss operates *only* on the identity features.
    - Because the new triplet loss calculates asymmetry using C (the dot product of identity features), the triplet loss backpropagates strong gradients directly into the identity backbone to satisfy the margin.
    - It is actively pushing positive identity features apart to manipulate C, which directly fights and worsens the alignment loss.

Uniform/Domain Loss Peaks & Poor Precision

- **Observation:** Both uniform and domain losses show a sudden peak before settling, and overall training precision remains poor.
- **Analysis (Latent Space Reorganization):** The peaks in the uniform and

domain losses indicate a sudden, violent reorganization of the latent space. As the network tries to push domains apart, the non-linear nature of the spherical tangents causes the embeddings to "snap" into a suboptimal configuration.

- **Analysis (Precision Collapse):** The poor training precision confirms that the model is failing to confidently classify identities. The theoretical strictness of the geodesic path has over-constrained the network.
- By forcing the drift to perfectly adhere to the hypersphere's surface, we removed the "slack" the network previously used in the Euclidean approximation to easily separate identities, resulting in a mathematically pure but empirically weaker representation.

[Sweep] Drift Vector Dimension Sweep

Previously:

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \sum_{k=1}^{S} \mathbf{x}_{\omega}^{(k)} (\mathbf{y}_{\text{id}}^{(k)} - \mathbf{x}_{\text{id}}^{(k)})$$

$$S = \min(\dim(\mathbf{x}_{\text{id}}), \dim(\mathbf{x}_{\omega}))$$

Now:

$$d_F(\mathbf{x}, \mathbf{y}) = \|\mathbf{x}_{\text{id}} - \mathbf{y}_{\text{id}}\|_2 + \frac{N}{2S} \sum_{k=1}^{S} \left(\mathbf{x}_{\omega}^{(k)} + \mathbf{y}_{\omega}^{(k)}\right) \left(\mathbf{y}_{\text{id}}^{(k)} - \mathbf{x}_{\text{id}}^{(k)}\right)$$

$$N = \dim(\mathbf{x}_{\text{id}})$$

$$S = \min(\dim(\mathbf{x}_{\text{id}}), \dim(\mathbf{x}_{\omega}))$$

- **Addressing previous feedback:** Investigated why reducing the drift dimension (S) from 2048 down to 16 seemed to artificially improve performance.
- **The Scaling Artifact:** Thomas' intuition was correct. The asymmetry term relies on a dot product. Summing over 16 elements instead of 2048 naturally produces a much smaller value, causing the distance function to degenerate back to the (better-performing) Euclidean baseline.

Why couldn't the network just learn larger weights to compensate? In a standard network, the optimizer would just scale up the ω vector. Here, the model is mathematically handcuffed by two constraints:

- **The Randers Constraint:** To keep the metric strictly positive definite, the L2-norm of the drift vector ($\| \omega \|$) is strictly capped at 0.95 by our FinslerDriftHead.
- **Identity Slicing:** The identity vector (y_{id} (is L2-normalized over all 2048 dimensions ($\| y_{id} \| = 1$).

Calculating the 11x Reduction:

- By the Cauchy-Schwarz inequality, the absolute maximum magnitude of our

asymmetry dot product is $\| \omega \| \times \| y_{id_slice} \|$.

- **At $S = 2048$:** We use the full identity vector. The max magnitude is $0.95 \times 1.0 = 0.95$.
- **At $S = 16$:** We only slice the first 16 dimensions of the normalized identity vector. Assuming features are roughly evenly distributed, the expected L2-norm of this 16-dim slice is $\sqrt{16/2048} \approx 0.088$.
- Therefore, the max magnitude at $S = 16$ is $0.95 \times 0.088 \approx 0.084$.
- Comparing the two maximums ($0.95 \div 0.084$), the asymmetry term at $S = 16$ is forced to be roughly **11 times smaller**, even when the model pushes the drift vector to its absolute limit.

The New Setup:

- To test if the previous improvements were purely this scaling artifact, we introduced the N/S scaling factor (where $N = 2048$).
- This artificially restores the magnitude of the lower-dimensional dot products to match the 2048-dim baseline.
- If the $S = 16$ model with this scaling performs poorly again, it proves the improvement was just an artifact of vanishing asymmetry, not genuinely better generalization.

EXTRA

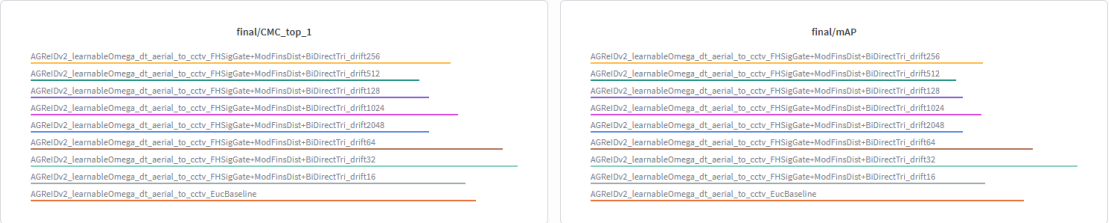
The Cauchy-Schwarz inequality is a fundamental principle in linear algebra that establishes an upper bound on the dot product (inner product) of two vectors based on their individual lengths (magnitudes).

The Formula

For any two vectors u and v, the inequality states:

$$| \langle u, v \rangle | \leq \| u \| \| v \|$$

[Sweep] Drift Vector Dimension Sweep



Rerun the experiment
Spherical interpolation with and without truncation

Hypothesis Confirmed: The introduction of the N/S scaling factor successfully decoupled the drift vector's dimensionality from the magnitude of the asymmetry term.

Flattened Performance Curve:

- Looking at the final/mAP and final/CMC_top_1 bar charts, the massive performance gap between lower dimensions ($S = 16$) and higher dimensions ($S = 2048$) has largely vanished.
- Previously, $S = 16$ artificially approached the Euclidean baseline. Now, with the asymmetry magnitude mathematically restored, $S = 16$ performs similarly to the rest of the pack (hovering around ~22-25% mAP).

The "Improvement" was an Illusion:

- Because lower dimensions no longer dominate when scaled fairly, we can conclude they were *not* learning a more generalizable representation.

- The previous success of $S = 16$ was purely a mathematical artifact: the asymmetry term became so small it effectively turned the model back into a standard Euclidean network.

Network Behavior (Omega Norms):

- The omega_norm_max and omega_norm_mean charts reveal that the network rapidly pushes the drift vector (ω) to its absolute ceiling (the 0.95 max_norm constraint).
- This indicates the optimizer is actively trying to maximize the drift magnitude to satisfy the loss functions, but is hard-capped by the strict Randers metric constraints.

Strategic Takeaway:

Next Steps: Instead of tuning the dimension size, we need to rethink *how* the asymmetry is applied or regularized. The current constrained drift might be too disruptive to the underlying identity manifold.

Regularization: -log(1-w); heuristic regularizers to prevent divergence of the omega norm

Or 1/(1-|w|) ; inside can be squared, outside too

[Master Thesis] Updates?

Master's Thesis Research Proposal

Target Group: Computer Vision Group (CVG), TUM
Date: February 18, 2026

Candidate: Lee Zhuo Le (03795610)
Program: M.Sc. Robotics, Cognition, Intelligence

Industrial Collaboration:
Isarsoft GmbH

Research Topic & Motivation

Proposed Title: Beyond Cosine: Geometric Metrics for Vision Foundation Model Embeddings

Problem Statement: While vision foundation models map images to $\mathbb{R}^d$, the standard practice of using cosine similarity implicitly assumes a flat Euclidean geometry. Recent studies show that while scaling laws and architectural optimizations like QK-Normalization have largely mitigated naive artifacts such as anisotropy [1] and "hubness" [2] in modern models [3], Euclidean metrics can limit downstream tasks. Explicit non-Euclidean metric modeling is required for tasks exhibiting asymmetric relationships and hierarchical logic [4], or multi-view spatial consistency [5]. Furthermore, current geometric assumptions

1. On Company "Inconsistency" (Why are they okay with different topics?)

- Isarsoft builds complex, association-heavy video pipelines.
- They have two core R&D needs: better tracking (ReID) and better feature extraction (Foundation Models).
- Both topics tackle their long-term technical bottlenecks. The flexibility isn't a lack of focus; it's a wide R&D net.

2. On "Added Value" (Knowledge Transfer vs. Product Features)

- I am not building a software feature or a product patch.
- I am generating foundational mathematical insights (e.g., neighborhood stability, retrieval logic).
- These insights inform their long-term engineering strategy, regardless of the specific code I write.

3. On "CVG in the Driver's Seat"

- **Stance:** Because their interests are broad, Isarsoft is intentionally deferring to **your** (CVG's) expertise.
- **Priority:** They do not want to force a corporate use case. They want the research direction that yields the strongest academic narrative and

publication chance.

4. On Resources (GPUs & Proprietary Data)

- **Agreement:** I fully agree with your feedback—the thesis must stand on its own using public benchmarks.
- **Pivot:** The company resources (RTX 6000s, proprietary footage) are strictly a **safety net** for optional robustness checks.
- **Promise:** They will not block open dissemination or dictate the methodology.

> The company is funding this position as a full-time role. This financial support allows me to dedicate 100% of my time to the thesis without the distraction of a separate part-time job. Being on-site also keeps me close to the problem space, even if the core methodology is purely academic.

Next Week

1. [**Distance**]: Use slerp interpolation to integrate on a spherical geodesic for option 2
   - Consider methods to learn omega on the tangent plane and enforce that
   - Example: Identity – XX^t is on the tangent of the sphere, where x is normalized
2. [**Training**]: Train the baseline and best model configuration with full 60 epochs
3. [**Training**]: Train the best model configuration on one of BAU's training dataset
   - Consider running a sweep with different random seeds to average out the results
4. [**Theory**]: Explain why drift align and bidirectional triplet are incompatible regularization terms together
5. [**Theory**]: Explain why having a 32-dimensional drift vector is optimal, compared to 64 and 16; and why the drift vector does not diverge to the max in this specific scenario
6. [**Model**]: Implement heuristic regularization terms (e.g., -log(1-w) or 1/(1-|w|)) to prevent divergence of the omega norm
7. [**Literature**]: Consult the SOTA for AGReIDv2 and the protocols from BAU, and benchmark the current top Finsler model against them. Decide or not to pursue publication given results.

[**Sanity Test**]: Devise a small experiment to test the logic behind cross-res asymmetry and camera-viewpoint bias (e.g., small dataset + simple metric).